

Лабораторная работа №3

Реализация динамического массива (аналог vector). Итераторы

Цель работы:

1. Реализовать шаблонный класс `Vector<T>` - аналог `std::vector` - с нуля
2. Освоить управление динамической памятью: выделение, расширение, освобождение
3. Закрепить «правило трёх» (деструктор, конструктор копирования, оператор присваивания) и освоить «правило пяти» (конструктор и оператор перемещения)
4. Реализовать итераторы для поддержки range-based for и STL-совместимости
5. Провести амортизированный анализ операции `push_back`

Теоретическое обоснование

1. Что такое динамический массив?

Динамический массив - это структура данных, которая хранит элементы в непрерывной области памяти, но в отличие от статического массива может изменять свой размер во время выполнения программы.

Характеристика	Статический массив	Динамический массив (vector)
Размер	Фиксированный (известен на этапе компиляции)	Изменяемый (определяется во время выполнения)
Память	Стек (ограничен)	Куча (heap) - практически неограничена
Расширение	Невозможно	Автоматическое (при переполнении)
Вставка в конец	Невозможно при заполнении	O(1) амортизированно

2. Основные компоненты класса Vector

```
template <typename T>  
class Vector {
```

private:

```
T* data;    // указатель на динамический массив
int sz;     // текущий размер (количество элементов)
int cap;    // ёмкость (выделенная память)
```

public:

// Конструкторы

```
Vector();           // по умолчанию
explicit Vector(int n); // размера n
Vector(int n, const T& val); // размера n, заполненный val
Vector(const Vector& other); // копирования
Vector(Vector&& other);    // перемещения
```

// Деструктор

```
~Vector();
```

// Операторы

```
Vector& operator=(const Vector& other);    // присваивание
копированием
```

```
Vector& operator=(Vector&& other);    // присваивание перемещением
T& operator[](int index);             // доступ по индексу
const T& operator[](int index) const; // константный доступ
```

// Методы доступа

```
int size() const;
int capacity() const;
bool empty() const;
```

// Методы изменения

```
void push_back(const T& value);
void pop_back();
void reserve(int newCap);
void resize(int newSize, const T& defaultValue = T());
void clear();
```

// Итераторы

```
T* begin();
T* end();
const T* begin() const;
const T* end() const;
```

```
};
```

3. Управление памятью

Выделение и освобождение

// Выделение памяти

```
data = new T[capacity];
```

// Освобождение памяти

```
delete[] data;
```

Расширение массива (стратегия удвоения)

```
void reserve(int newCap) {  
    if (newCap <= cap) return;  
  
    T* newData = new T[newCap];  
    for (int i = 0; i < sz; i++) {  
        newData[i] = data[i];  
    }  
    delete[] data;  
    data = newData;  
    cap = newCap;  
}
```

Амортизированный анализ push_back

Стратегия удвоения ёмкости обеспечивает амортизированную сложность $O(1)$:

Действие	Количество копирований
Вставка 1	0
Вставка 2 (расширение 1→2)	1
Вставка 3	0
Вставка 4 (расширение 2→4)	2
Вставки 5-7	0
Вставка 8 (расширение 4→8)	4

Итог: при n вставках выполняется около $2n$ копирований $\rightarrow O(n)$ на n операций $\rightarrow O(1)$ амортизированно на одну вставку.

4. Правило трёх и правило пяти

Правило	Методы
Правило трёх	Деструктор, конструктор копирования, оператор присваивания копированием
Правило пяти	+ Конструктор перемещения, оператор присваивания перемещением

Конструктор копирования (глубокое копирование)

```
Vector(const Vector& other) : sz(other.sz), cap(other.cap) {  
    data = new T[cap];  
    for (int i = 0; i < sz; i++) {  
        data[i] = other.data[i];  
    }  
}
```

Оператор присваивания (с защитой от самоприсваивания)

```
Vector& operator=(const Vector& other) {  
    if (this != &other) {  
        Vector temp(other); // copy-and-swap idiom  
        swap(temp);  
    }  
    return *this;  
}
```

Конструктор перемещения

```
Vector(Vector&& other) noexcept  
: data(other.data), sz(other.sz), cap(other.cap) {  
    other.data = nullptr;  
    other.sz = 0;  
    other.cap = 0;  
}
```

5. Итераторы

Итератор - объект, который указывает на элемент контейнера и позволяет перебирать элементы последовательно.

```
// Простая реализация итератора (указатель)
T* begin() { return data; }
T* end() { return data + sz; }
```

```
const T* begin() const { return data; }
const T* end() const { return data + sz; }
```

```
// Использование
```

```
Vector<int> v;
for (int x : v) {           // range-based for
    cout << x << " ";
}

for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

6. Полный пример: реализация Vector

```
#include <iostream>
#include <stdexcept>
```

```
template <typename T>
class Vector {
private:
    T* data;
    int sz;
    int cap;
```

```
public:
```

```
    // Конструктор по умолчанию
```

```
    Vector() : data(nullptr), sz(0), cap(0) {}
```

```
    // Конструктор размера
```

```
    explicit Vector(int n) : sz(n), cap(n) {
        data = new T[cap];
        for (int i = 0; i < sz; i++) {
            data[i] = T();
        }
    }
```

```
    // Конструктор копирования
```

```
    Vector(const Vector& other) : sz(other.sz), cap(other.cap) {
```

```

        data = new T[cap];
        for (int i = 0; i < sz; i++) {
            data[i] = other data[i];
        }
    }

// Конструктор перемещения
Vector(Vector&& other) noexcept
    : data(other data), sz(other sz), cap(other cap) {
    other data = nullptr;
    other sz = 0;
    other cap = 0;
}

// Деструктор
~Vector() {
    delete[] data;
}

// Оператор присваивания копированием
Vector& operator=(const Vector& other) {
    if (this != &other) {
        Vector temp(other);
        swap(temp);
    }
    return *this;
}

// Оператор присваивания перемещением
Vector& operator=(Vector&& other) noexcept {
    if (this != &other) {
        delete[] data;
        data = other data;
        sz = other sz;
        cap = other cap;
        other data = nullptr;
        other sz = 0;
        other cap = 0;
    }
    return *this;
}

```

```
// Доступ по индексу
T& operator[](int index) {
    if (index < 0 || index >= sz) {
        throw std::out_of_range("Index out of range");
    }
    return data[index];
}
```

```
const T& operator[](int index) const {
    if (index < 0 || index >= sz) {
        throw std::out_of_range("Index out of range");
    }
    return data[index];
}
```

```
// Методы доступа
int size() const { return sz; }
int capacity() const { return cap; }
bool empty() const { return sz == 0; }
```

```
// Резервирование памяти
void reserve(int newCap) {
    if (newCap <= cap) return;

    T* newData = new T[newCap];
    for (int i = 0; i < sz; i++) {
        newData[i] = data[i];
    }
    delete[] data;
    data = newData;
    cap = newCap;
}
```

```
// Добавление в конец
void push_back(const T& value) {
    if (sz >= cap) {
        int newCap = (cap == 0) ? 1 : cap * 2;
        reserve(newCap);
    }
    data[sz++] = value;
}
```

// Удаление последнего элемента

```
void pop_back() {  
    if (empty()) {  
        throw std::underflow_error("Vector is empty");  
    }  
    sz--;  
}
```

// Изменение размера

```
void resize(int newSize, const T& defaultValue = T()) {  
    if (newSize < sz) {  
        sz = newSize;  
    } else if (newSize > sz) {  
        if (newSize > cap) {  
            reserve(newSize);  
        }  
        for (int i = sz; i < newSize; i++) {  
            data[i] = defaultValue;  
        }  
        sz = newSize;  
    }  
}
```

// Очистка

```
void clear() {  
    sz = 0;  
}
```

// Обмен содержимого

```
void swap(Vector& other) {  
    std::swap(data, other.data);  
    std::swap(sz, other.sz);  
    std::swap(cap, other.cap);  
}
```

// Итераторы

```
T* begin() { return data; }  
T* end() { return data + sz; }  
const T* begin() const { return data; }  
const T* end() const { return data + sz; }  
};
```

```

// Демонстрация работы
int main() {
    setlocale(LC_ALL, "ru");

    Vector<int> v;

    cout << "=== Демонстрация Vector ===" << endl;
    cout << "Начальный размер: " << v.size() << ", ёмкость: " << v.capacity()
<< endl;

    for (int i = 1; i <= 10; i++) {
        v.push_back(i);
        cout << "push_back(" << i << "): размер=" << v.size()
            << ", ёмкость=" << v.capacity() << endl;
    }

    cout << "\nСодержимое через range-based for: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;

    cout << "\nДоступ по индексу: v[5] = " << v[5] << endl;

    v.pop_back();
    cout << "\nПосле pop_back: размер=" << v.size() << endl;

    v.resize(15, 0);
    cout << "После resize(15, 0): размер=" << v.size() << endl;

    return 0;
}

```

Примеры выполнения практической части

Пример 1. Базовые операции Vector (базовый уровень)

Задача: Реализовать класс Vector с основными методами: конструкторы, push_back, pop_back, operator[], size.

```

#include <iostream>

template <typename T>
class Vector {

```

private:

```
T* data;  
int sz;  
int cap;
```

public:

```
Vector() : data(nullptr), sz(0), cap(0) {}
```

```
explicit Vector(int n) : sz(n), cap(n) {  
    data = new T[cap];  
    for (int i = 0; i < sz; i++) data[i] = T();  
}
```

```
~Vector() { delete[] data; }
```

```
void push_back(const T& value) {  
    if (sz >= cap) {  
        int newCap = (cap == 0) ? 1 : cap * 2;  
        T* newData = new T[newCap];  
        for (int i = 0; i < sz; i++) newData[i] = data[i];  
        delete[] data;  
        data = newData;  
        cap = newCap;  
    }  
    data[sz++] = value;  
}
```

```
void pop_back() { if (sz > 0) sz--; }
```

```
int size() const { return sz; }
```

```
int capacity() const { return cap; }
```

```
T& operator[](int index) { return data[index]; }
```

```
const T& operator[](int index) const { return data[index]; }
```

```
void print() const {  
    std::cout << "[";  
    for (int i = 0; i < sz; i++) {  
        std::cout << data[i];  
        if (i < sz - 1) std::cout << ", ";  
    }  
    std::cout << "]";
```

```

    }
};

int main() {
    Vector<int> v;

    for (int i = 1; i <= 5; i++) {
        v.push_back(i * 10);
    }

    std::cout << "Размер: " << v.size() << ", Ёмкость: " << v.capacity() <<
std::endl;
    std::cout << "Элементы: ";
    v.print();
    std::cout << std::endl;

    v.pop_back();
    std::cout << "После pop_back: ";
    v.print();
    std::cout << std::endl;

    return 0;
}

```

Пример 2. Итераторы (средний уровень)

Задача: Добавить в класс Vector итераторы и продемонстрировать их использование.

```

#include <iostream>
#include <algorithm>

template <typename T>
class Vector {
private:
    T* data;
    int sz;
    int cap;

public:
    // ... остальные методы ...

    // Итераторы
    T* begin() { return data; }

```

```
T* end() { return data + sz; }
const T* begin() const { return data; }
const T* end() const { return data + sz; }
```

```
// Вставка по итератору
```

```
void insert(T* pos, const T& value) {
    int index = pos - data;
    if (index < 0 || index > sz) return;

    if (sz >= cap) {
        int newCap = (cap == 0) ? 1 : cap * 2;
        T* newData = new T[newCap];
        for (int i = 0; i < index; i++) newData[i] = data[i];
        newData[index] = value;
        for (int i = index; i < sz; i++) newData[i + 1] = data[i];
        delete[] data;
        data = newData;
        cap = newCap;
    } else {
        for (int i = sz; i > index; i--) data[i] = data[i - 1];
        data[index] = value;
    }
    sz++;
}
```

```
// Удаление по итератору
```

```
void erase(T* pos) {
    int index = pos - data;
    if (index < 0 || index >= sz) return;

    for (int i = index; i < sz - 1; i++) data[i] = data[i + 1];
    sz--;
}

};
```

```
int main() {
    setlocale(LC_ALL, "ru");
```

```
    Vector<int> v;
```

```
    for (int i = 1; i <= 5; i++) {
        v.push_back(i);
```

```

    }

    std::cout << "Исходный вектор: ";
    for (int x : v) std::cout << x << " ";
    std::cout << std::endl;

    // Вставка в середину
    auto it = v.begin() + 2;
    v.insert(it, 99);

    std::cout << "После вставки 99 на позицию 2: ";
    for (int x : v) std::cout << x << " ";
    std::cout << std::endl;

    // Удаление
    it = v.begin() + 3;
    v.erase(it);

    std::cout << "После удаления элемента на позиции 3: ";
    for (int x : v) std::cout << x << " ";
    std::cout << std::endl;

    // Сортировка с использованием STL алгоритма
    std::sort(v.begin(), v.end());
    std::cout << "После сортировки: ";
    for (int x : v) std::cout << x << " ";
    std::cout << std::endl;

    return 0;
}

```

Пример 3. Амортизированный анализ (продвинутый уровень)

Задача: Реализовать класс `Vector` с возможностью отслеживания количества операций и построить график зависимости времени от размера.

```

#include <iostream>
#include <chrono>
#include <vector>
#include <iomanip>

template <typename T>
class Vector {
private:

```

```

T* data;
int sz;
int cap;
int copyCount; // счётчик копирований

public:
    Vector() : data(nullptr), sz(0), cap(0), copyCount(0) {}

    ~Vector() { delete[] data; }

    void push_back(const T& value) {
        if (sz >= cap) {
            int newCap = (cap == 0) ? 1 : cap * 2;
            T* newData = new T[newCap];
            for (int i = 0; i < sz; i++) {
                newData[i] = data[i];
                copyCount++;
            }
            delete[] data;
            data = newData;
            cap = newCap;
        }
        data[sz++] = value;
    }

    int getCopyCount() const { return copyCount; }
    int size() const { return sz; }
    int capacity() const { return cap; }

    void resetCopyCount() { copyCount = 0; }
};

int main() {
    setlocale(LC_ALL, "ru");

    std::cout << "=== Амортизированный анализ push_back ===" <<
std::endl
    std::cout << std::setw(10) << "n"
        << std::setw(15) << "Копирований"
        << std::setw(15) << "Всего операций"
        << std::setw(15) << "Среднее" << std::endl;
    std::cout << std::string(55, '-') << std::endl;

```

```

for (int n = 1000; n <= 100000; n *= 10) {
    Vector<int> v;

    for (int i = 0; i < n; i++) {
        v.push_back(i);
    }

    int totalOps = n + v.getCopyCount();
    double average = static_cast<double>(totalOps) / n;

    std::cout << std::setw(10) << n
                << std::setw(15) << v.getCopyCount()
                << std::setw(15) << totalOps
                << std::setw(15) << std::fixed << std::setprecision(2) << average
                << std::endl;
}

return 0;
}

```

Ожидаемый вывод:

=== Амортизированный анализ push_back ===

n	Копирований	Всего операций	Среднее
1000	1023	2023	2.02
10000	10239	20239	2.02
100000	131071	131071	1.31

Индивидуальные задания

Базовый уровень (15 вариантов)

Общее задание: Реализовать класс Vector с минимальным набором методов согласно варианту.

№ вар.	Конструкторы	Обязательные методы
1	По умолчанию, с параметром (размер)	push_back, pop_back, size, capacity
2	По умолчанию, с параметром (размер)	push_back, pop_back, empty, clear

№ вар.	Конструкторы	Обязательные методы
3	По умолчанию, с параметром (размер, значение)	push_back, size, operator[], print
4	По умолчанию, копирования	push_back, pop_back, size, reserve
5	По умолчанию, с параметром (размер)	push_back, insert, erase, size
6	По умолчанию, с параметром (размер, значение)	push_back, pop_back, find, print
7	По умолчанию, с параметром (размер)	push_back, pop_back, resize, capacity
8	По умолчанию, копирования	push_back, operator[], size, clear
9	По умолчанию, с параметром (размер)	push_back, pop_back, front, back
10	По умолчанию, с параметром (размер, значение)	push_back, size, empty, reserve
11	По умолчанию, с параметром (размер)	push_back, pop_back, insert, print
12	По умолчанию, копирования	push_back, erase, size, capacity
13	По умолчанию, с параметром (размер)	push_back, pop_back, resize, print
14	По умолчанию, с параметром (размер, значение)	push_back, size, operator[], clear

№ вар.	Конструкторы	Обязательные методы
15	По умолчанию, с параметром (размер)	push_back, pop_back, front, back, size

Средний уровень (15 вариантов)

Общее задание: Реализовать класс Vector с полной поддержкой «правила пяти» и дополнительной функциональностью согласно варианту.

№ вар.	Дополнительная функциональность
1	Методы reserve, shrink_to_fit
2	Метод reverse() - разворот массива
3	Методы sum, average (для числовых типов)
4	Методы min, max
5	Методы find (поиск по значению), contains
6	Метод removeAll (удаление всех вхождений значения)
7	Метод slice(int start, int end) - создание подвектора
8	Метод concat(const Vector& other) - объединение векторов
9	Метод swap (обмен содержимым)
10	Метод unique - удаление дубликатов
11	Метод rotate(int k) - циклический сдвиг
12	Метод shuffle - случайное перемешивание
13	Метод sort - сортировка пузырьком
14	Метод merge - слияние двух отсортированных векторов
15	Метод isSorted - проверка на упорядоченность

Продвинутый уровень (15 вариантов)

Общее задание: Реализовать класс `Vector` с поддержкой «правила пяти», итераторами и дополнительной функциональностью согласно варианту.

№ вар.	Дополнительная функциональность
1	Итераторы с категориями (input, output, forward, bidirectional, random access)
2	Реализация <code>const_iterator</code>
3	Реализация <code>reverse_iterator</code> и методов <code>rbegin</code> , <code>rend</code>
4	Операторы сравнения векторов (<code>==</code> , <code>!=</code> , <code><</code> , <code>></code>)
5	Операторы <code>+</code> (конкатенация) и <code>*</code> (умножение на число)
6	Метод <code>apply(function)</code> - применение функции ко всем элементам
7	Метод <code>filter(predicate)</code> - фильтрация элементов
8	Сериализация: <code>saveToFile</code> и <code>loadFromFile</code>
9	Поддержка пользовательских аллокаторов
10	Метод <code>partition(pivot)</code> - разделение на две части
11	Реализация быстрой сортировки внутри класса
12	Ленивое копирование (copy-on-write)
13	Реализация <code>std::initializer_list</code> для удобной инициализации
14	Реализация <code>emplace_back</code> (вариативный шаблон)
15	Реализация оператора <code><<</code> для вывода вектора

Контрольные вопросы

1. Почему динамический массив называется «динамическим»?
2. Что такое ёмкость (`capacity`) и чем она отличается от размера (`size`)?

3. Почему при расширении массива ёмкость удваивается, а не увеличивается на фиксированное число?
4. Какова амортизированная сложность операции `push_back`? Почему?
5. Что такое «правило трёх»? Когда оно применяется?
6. Что такое «правило пяти»? Чем оно отличается от «правила трёх»?
7. В чём разница между конструктором копирования и оператором присваивания?
8. Зачем нужна проверка `if (this != &other)` в операторе присваивания?
9. Что такое итератор? Как он связан с указателем?
10. Зачем нужны методы `begin()` и `end()`?
11. Как реализовать `range-based for` для своего контейнера?
12. Что такое «глубокое копирование»? Чем оно отличается от «поверхностного»?
13. Что такое семантика перемещения и зачем она нужна?
14. Почему конструктор перемещения помечается `noexcept`?
15. Как правильно освободить память в деструкторе?

Требования к отчёту

Титульный лист с названием работы, ФИО студента, номером группы, вариантом

Цель работы (формулируется самостоятельно на основе предложенной)

Теоретическая часть (основные понятия: динамическая память, правило трёх/пяти, итераторы, амортизированный анализ)

Постановка задачи (текст задания согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода программы)

Анализ сложности (для каждой операции указать сложность)

Ответы на контрольные вопросы (письменно, развёрнуто)

Выводы (что нового узнали, какие навыки приобрели)

Ссылка на Git-репозиторий с кодом программы

Критерии оценивания

Базовый уровень (максимум 60 баллов)

Критерий	Баллы
Класс <code>Vector</code> реализован корректно, поля приватные	10

Критерий	Баллы
Реализованы конструкторы (по умолчанию, с параметром)	10
Реализован деструктор, освобождает память	10
Реализованы указанные в варианте методы	15
Демонстрационная программа показывает работу всех методов	10
Код отформатирован, есть комментарии	5

Средний уровень (максимум 80 баллов)

Критерий	Баллы
Все требования базового уровня	40
Реализовано «правило пяти»	15
Реализована дополнительная функциональность по варианту	15
Корректная обработка ошибок (выход за границы)	5
Наличие проверки на самоприсваивание	5

Продвинутый уровень (максимум 100 баллов)

Критерий	Баллы
Все требования среднего уровня	60
Реализация итераторов (begin, end)	10
Реализация продвинутой функциональности по варианту	15
Исключения и безопасность (RAII)	10
Наличие ссылки на Git-репозиторий	5

Штрафные баллы

Нарушение	Штраф
Утечка памяти (не освобождена динамическая память)	-20
Двойное освобождение памяти	-15
Отсутствие проверки на самоприсваивание	-5
Использование публичных полей вместо геттеров	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка (традиционная)	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60